

FACULDADE DE TECNOLOGIA SENAI FLORIANÓPOLIS
CURSO DE GRADUAÇÃO TECNOLÓGICA EM BACHARELADO EM CIÊNCIA
DE DADOS E INTELIGÊNCIA ARTIFICIAL

Sistema IoT de Manutenção Preditiva em Tempo Real de Engenharia de Dados e
MLOPs

Aluna: Nyrx Oliveira de Aquino Farias

UniSENAI Florianópolis

19/08/2026

1. Seção 1 - Respostas teóricas e analíticas (Questões , 2 e 3).

→ **Questão 1** - Arquitetura de Roteamento de Modelos (Teórica) Observe a função `prever_ponto_dinamicamente()`. Explique como o código decide qual modelo de Machine Learning (`modelo_rodando` ou `modelo_parado`) deve ser acionado a cada nova leitura. Por que essa abordagem de "chaveamento dinâmico" baseada na variável `status_operacao` é uma boa prática em sistemas de monitoramento industrial em vez de usar um único modelo global?

Resposta:

- **Como o código decide qual modelo acionar a cada leitura?**

A função `prever_ponto_dinamicamente()` recebe um dicionário contendo os dados do sensor e extrai o valor da variável `status_operacao`. Em seguida, verifica se esse valor pertence à lista `status_motor_rodando = [2, 7, 8, 9, 10, 11, 12, 13, 14]`. Quando o status está presente nessa lista, o sistema considera que o motor está em operação e aciona o modelo `modelo_motor_rodando.joblib`. Esse modelo utiliza as variáveis operacionais dos sensores e horímetros, além das features derivadas pelo simulador, como `_media_7dias`, `_std_7dias` e `_desvio_7dias`, para realizar a previsão de possíveis falhas nos próximos sete dias. Quando o status não pertence à lista, o sistema considera que o motor está parado ou inativo e, nesse caso, aciona o modelo `modelo_motor_parado.joblib`. Esse segundo modelo utiliza as variáveis contínuas dos sensores e os horímetros definidos como entradas para o modelo de motor parado, sem as features derivadas de sete dias utilizadas pelo modelo de motor rodando.

- **Por que o chaveamento dinâmico pode ser mais adequado que um modelo global único?**

O chaveamento dinâmico pode ser mais adequado porque o equipamento apresenta comportamentos diferentes de acordo com seu estado operacional. Um compressor em funcionamento pode apresentar padrões de pressão, temperatura, vazão e corrente diferentes daqueles observados quando está parado. Dessa forma, a utilização de modelos especializados permite que cada um seja desenvolvido para um contexto específico, evitando que um único modelo precise aprender simultaneamente comportamentos de regimes distintos. Além disso, as distribuições estatísticas das variáveis podem variar entre os estados de operação e inatividade, o que pode dificultar a aprendizagem de um modelo global. O uso de conjuntos de features específicos para cada contexto também contribui para uma representação mais adequada dos dados, já que o modelo de motor rodando utiliza variáveis derivadas relacionadas ao comportamento operacional, enquanto o modelo de motor parado trabalha com outro conjunto de entradas. Outro benefício é a especialização dos modelos, permitindo que cada um aprenda padrões associados ao respectivo regime operacional e possa ser ajustado de forma independente. Em uma arquitetura de MLOps, essa separação também facilita o treinamento, a validação, o monitoramento e a atualização individual dos modelos, de modo que uma alteração ou melhoria no modelo utilizado para o motor rodando não exija necessariamente modificações no modelo destinado ao motor parado.

→ **Questão 2** - Análise Crítica de Engenharia de Recursos (Feature Engineering) No código, o cálculo de variáveis derivadas (como `_media_7dias`, `_std_7dias` e `_desvio_7dias`) é feito multiplicando o dado bruto instantâneo por fatores fixos (ex: 0.95 ou 0.1). a) Quais são os problemas técnicos e operacionais de realizar essa aproximação em um ambiente de produção real ? b) Como uma arquitetura de dados em nuvem ideal deveria calcular esses agregados temporais de 7 dias antes de enviar o vetor para o modelo?

Resposta:

a) *Quais são os problemas técnicos e operacionais de realizar essa aproximação em um ambiente de produção real ?*

Problemas da aproximação por fatores fixos (0,95 e 0,1). No código, as variáveis derivadas são calculadas a partir do valor instantâneo do sensor, utilizando fatores fixos, como $_media_7dias = valor_bruto \times 0,95$, $_std_7dias = valor_bruto \times 0,1$ e $_desvio_7dias = valor - (valor \times 0,95)$. Essa abordagem apresenta limitações importantes para um ambiente de produção, pois não representa o comportamento histórico real do equipamento. A chamada “média de sete dias”, por exemplo, não consulta nenhuma leitura anterior, sendo apenas uma transformação do valor atual. Considerando a frequência de uma leitura a cada dez minutos proposta pelo cenário, uma janela de sete dias teria aproximadamente 1.008 leituras, que deveriam ser utilizadas no cálculo da média e do desvio-padrão reais.

Outra limitação é a redundância de informação. Como as variáveis derivadas são determinadas diretamente a partir do valor instantâneo por meio de constantes fixas, elas não acrescentam informações temporais reais ao modelo. Além disso, essa aproximação prejudica a capacidade de identificar anomalias. Por exemplo, caso a pressão passe subitamente de 10 para 50 bar, a média aproximada seria imediatamente calculada como 47,5 bar, enquanto uma média real baseada nas leituras dos sete dias anteriores tenderia a permanecer mais próxima dos valores históricos e, justamente por isso, evidenciaria a mudança brusca do comportamento.

Em um ambiente de produção, essa estratégia também pode gerar inconsistências entre treinamento e inferência. Caso o modelo tenha sido treinado utilizando agregados temporais calculados a partir de históricos reais, mas durante a inferência receba valores aproximados por fatores fixos, haverá uma diferença entre a distribuição das features utilizadas no treinamento e aquela observada em produção, caracterizando um possível problema de training-serving skew e podendo prejudicar o desempenho do modelo.

b) *Como uma arquitetura de dados em nuvem ideal deveria calcular esses agregados temporais de 7 dias antes de enviar o vetor para o modelo?*

Utilizando Arquitetura em nuvem para cálculo dos agregados temporais. Em uma arquitetura de produção, os dados de telemetria deveriam ser recebidos continuamente por uma camada de ingestão, utilizando, por exemplo, Apache Kafka ou Amazon Kinesis. Esses dados seriam encaminhados para uma camada de processamento de fluxo, como Apache Flink ou Spark Structured Streaming, responsável por manter janelas temporais deslizantes de sete dias e calcular agregados reais, como média, desvio-padrão, mínimo, máximo e tendências das variáveis monitoradas. As features resultantes poderiam ser disponibilizadas em uma Feature Store, como Feast ou SageMaker Feature Store, garantindo consistência entre treinamento e inferência e permitindo acesso de baixa latência durante a predição. Paralelamente, as leituras brutas deveriam ser preservadas em um Data Lake ou banco especializado em séries temporais para auditoria, análises históricas e futuros retreinamentos. Por fim, o serviço de inferência receberia o vetor de features calculado a partir dos dados históricos e enviaria essas informações ao modelo de Machine Learning para gerar a previsão.

→ **Questão 3** - Simulação de Latência e Gargalos no Streamlit O script utiliza o parâmetro `tempo_de_espera = intervalo_base_segundos + random.uniform(0, atraso_iot_max_segundos)` junto com `time.sleep()`. a) Qual fenômeno do mundo real da comunicação IoT essa linha tenta simular? b) O uso do laço `while st.session_state.running:` combinado com `time.sleep()` pode travar a renderização da interface do Streamlit sob certas condições. Qual estratégia ou componente do Streamlit poderia ser utilizado para atualizar a tela sem bloquear a thread principal?

Resposta:

a) Qual fenômeno do mundo real da comunicação IoT essa linha tenta simular?

A expressão `tempo_de_espera = intervalo_base_segundos + random.uniform(0, atraso_iot_max_segundos)`, seguida de `time.sleep(tempo_de_espera)`, simula a variabilidade no intervalo de transmissão e na latência de comunicação dos dispositivos IoT. No código, o `intervalo_base_segundos` representa o período esperado entre o envio de duas leituras consecutivas, enquanto o valor aleatório acrescentado pelo `random.uniform()` representa uma variação adicional nesse tempo. Dessa forma, em vez de as mensagens serem recebidas sempre exatamente no mesmo intervalo, cada leitura pode sofrer um pequeno atraso diferente. Essa característica aproxima a simulação do comportamento de redes IoT reais, nas quais a comunicação pode apresentar variações devido à qualidade do sinal, interferências eletromagnéticas, retransmissões de pacotes, congestionamento da rede, processamento em gateways e oscilações na latência da infraestrutura de comunicação. No contexto da atividade, o valor de 10 segundos utilizado pelo simulador representa uma escala reduzida para simular o intervalo de aproximadamente 10 minutos indicado no cenário, tornando a execução mais rápida para fins de demonstração.

b) Qual estratégia ou componente do Streamlit poderia ser utilizado para atualizar a tela sem bloquear a thread principal?

O uso do laço `while st.session_state.running` combinado com `time.sleep()` pode prejudicar a responsividade da aplicação porque a execução permanece presa no laço durante o período de espera. Enquanto o `sleep()` está em execução, novas interações do usuário podem ficar atrasadas e a interface pode deixar de responder de maneira adequada. Uma alternativa mais apropriada é utilizar o recurso `st.fragment` do Streamlit com o parâmetro `run_every`, permitindo que apenas o trecho responsável pela atualização dos dados seja executado novamente em intervalos definidos, sem a necessidade de manter um laço infinito com `time.sleep()`. Essa abordagem é particularmente adequada para aplicações de monitoramento e streaming, pois possibilita atualizar periodicamente apenas uma parte da interface, mantendo o restante da aplicação disponível para interação. Outra possibilidade é utilizar um componente de atualização automática, como `streamlit-autorefresh`, que provoca novas execuções da aplicação em intervalos definidos sem recorrer a uma espera bloqueante. Em um ambiente de produção, uma solução ainda mais robusta seria separar a coleta e o processamento da telemetria da camada de visualização, mantendo o recebimento dos dados em um serviço independente e fazendo com que o Streamlit apenas consulte ou receba os resultados processados. Dessa maneira, a aquisição contínua dos dados não ficaria diretamente vinculada ao ciclo de renderização da interface.

2. Seção 2 - Desafio Prático de Código

- **Questão 4** - Modifique o código-fonte fornecido para adicionar uma Tabela de Histórico Recente na interface do Streamlit. Requisitos da implementação: 1. Armazene as últimas 5 leituras recebidas dentro da sessão do Streamlit (`st.session_state`). 2. Exiba uma tabela (`st.dataframe` ou `st.table`) logo abaixo dos cartões de status contendo: o Horário da leitura, o Status de Operação, o Valor de pressão_saida, o Resultado da Predição (Normal ou Possível Falha). (Cole no relatório o trecho do código alterado e inclua um print da tela do aplicativo rodando com a nova funcionalidade).

Para atender ao requisito da Questão 4, o código da aplicação `Teste_Compressor.py` foi modificado para armazenar as cinco leituras mais recentes utilizando `st.session_state` e exibi-las em uma tabela por meio do componente `st.dataframe()`. A implementação foi realizada de forma que cada nova leitura seja inserida no início do histórico, mantendo as informações mais recentes no topo e descartando automaticamente os registros excedentes.

★ Inicialização do histórico

Foi adicionada a importação de ***datetime*** para registrar o horário de cada leitura. Em seguida, foi criada uma lista em `st.session_state` para armazenar o histórico durante a sessão da aplicação.

Código em Python

```
from datetime import datetime

st.markdown("----")
st.markdown("### Tabela de Histórico Recente (Últimas 5 Leituras)")
placeholder_historico = st.empty()

if 'historico_leituras' not in st.session_state:
    st.session_state.historico_leituras = []
```

★ Registro de cada nova leitura

Após a geração da predição, são organizados os dados que devem compor o histórico. O status numérico recebido pelo simulador é convertido para uma descrição textual, enquanto o resultado do modelo é classificado como Normal ou Possível Falha.

Código em Python

```
horario_leitura = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

status_texto = (
    "Rodando"
    if current_status in status_motor_rodando
    else "Parado"
)

resultado_predicao = (
    "Possível Falha"
    if "Falha" in label
    else "Normal"
)

nova_leitura = {
    'Horário': horario_leitura,
    'Status Operação': status_texto,
    'Pressão Saída': round(novo_ponto['pressao_saida'], 2),
    'Predição': resultado_predicao
}
```

★ Controle das cinco últimas leituras

A nova leitura é inserida no início da lista para que os registros mais recentes apareçam primeiro. Em seguida, a lista é limitada a cinco elementos.

Código em Python

```
st.session_state.historico_leituras.insert(
    0,
    nova_leitura
)

if len(st.session_state.historico_leituras) > 5:
    st.session_state.historico_leituras = (
        st.session_state.historico_leituras[:5]
    )
```

★ Exibição da tabela

Por fim, o histórico armazenado no `st.session_state` é convertido em um `DataFrame` e apresentado na interface do Streamlit.

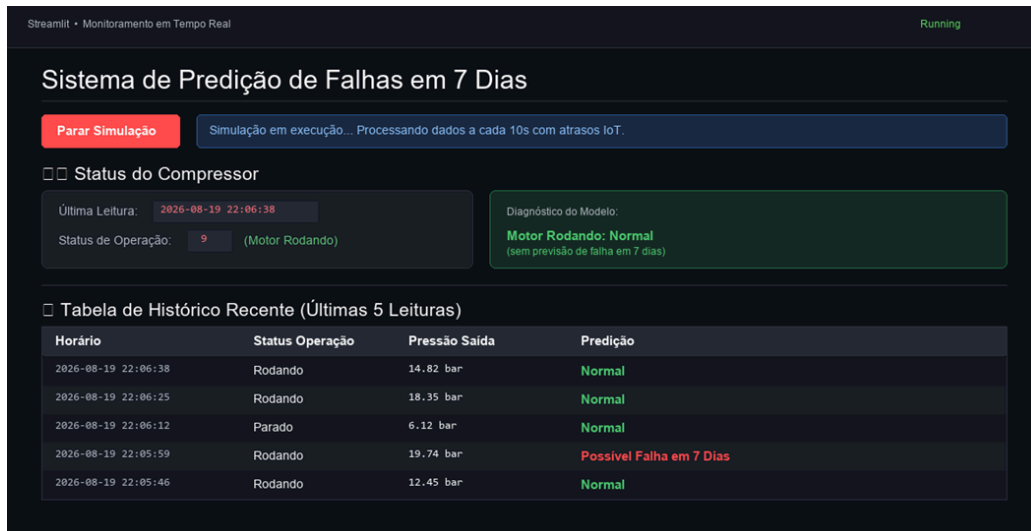
Código em Python

```
with placeholder_historico.container():
    df_historico = pd.DataFrame(
        st.session_state.historico_leituras
    )

    st.dataframe(
        df_historico,
        use_container_width=True,
        hide_index=True
    )
```

★ Comprovação de execução no navegador

A imagem a seguir demonstra a aplicação Streamlit em execução no navegador, com a simulação ativa e a Tabela de Histórico Recente preenchida. A tabela apresenta as cinco últimas leituras recebidas, contendo o horário da leitura, o status de operação, o valor de pressão de saída e o resultado da predição, conforme especificado nos requisitos da atividade.



Código em Python dos recortes da 4

```
# Importação adicional
(topo do arquivo) from
datetime import datetime

# Placeholder e inicialização do histórico (após os
placeholders de status) st.markdown("---")
st.markdown("### Tabela de Histórico Recente (Últimas 5
Leituras)") placeholder_historico = st.empty()

if 'historico_leituras' not in st.session_state:
    st.session_state.historico_leituras = []

# Dentro do loop de simulação (após a predição e
exibição dos cartões) horario_leitura =
datetime.now().strftime('%Y-%m-%d %H:%M:%S')
status_texto = "Rodando" if current_status in
status_motor_rodando else "Parado" resultado_predicao =
"Possível Falha" if "Falha" in label else "Normal"

nova_leitura = {
    'Horário':
        horario_leitura,
    'Status
Operação':
        status_texto,
    'Pressão Saída':
        round(novo_ponto['pressao_saida'], 2),
    'Predição': resultado_predicao
}

# Insere no início da lista (leituras mais recentes
primeiro) st.session_state.historico_leituras.insert(0,
nova_leitura)

# Mantém apenas as últimas 5 leituras
```

```
if len(st.session_state.historico_leituras) > 5:
    st.session_state.historico_leituras =
        st.session_state.historico_leituras[:5]

# Exibe a tabela atualizada
with placeholder_historico.container():
    df_historico =
        pd.DataFrame(st.session_state.historico_leituras)
    st.dataframe(df_historico, use_container_width=True,
        hide_index=True)
```

Link do Github

- <https://github.com/NyrxScar/Aula-3-Sistema-IoT-de-Manuten-o-Preditiva-de-Engenharia-de-Dados-e-MLOPs.git>